Computer Networks

CSA0712

CO4_Assessment Tool_1

Code Challenge

Presented by: C. Sravanthi

192525463

Question 1 — TCP Client–Server Application

Aim:

To develop a TCP client-server application using Python in which the client sends messages and the server processes and responds to them until the client sends "exit".

Server Program

```
  import socket
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind(("127.0.0.1", 5000))
server_socket.listen(1)
print("Server is waiting for connection...")
conn, addr = server_socket.accept()
print("Connected with:", addr)
while True:
    message = conn.recv(1024).decode()
if message == "exit":
        print("Client disconnected.")
        break
print("Client:", message)
 response = "Server received: " + message
```

```
conn.send(response.encode())
conn.close()
server_socket.close()
```

Client Program

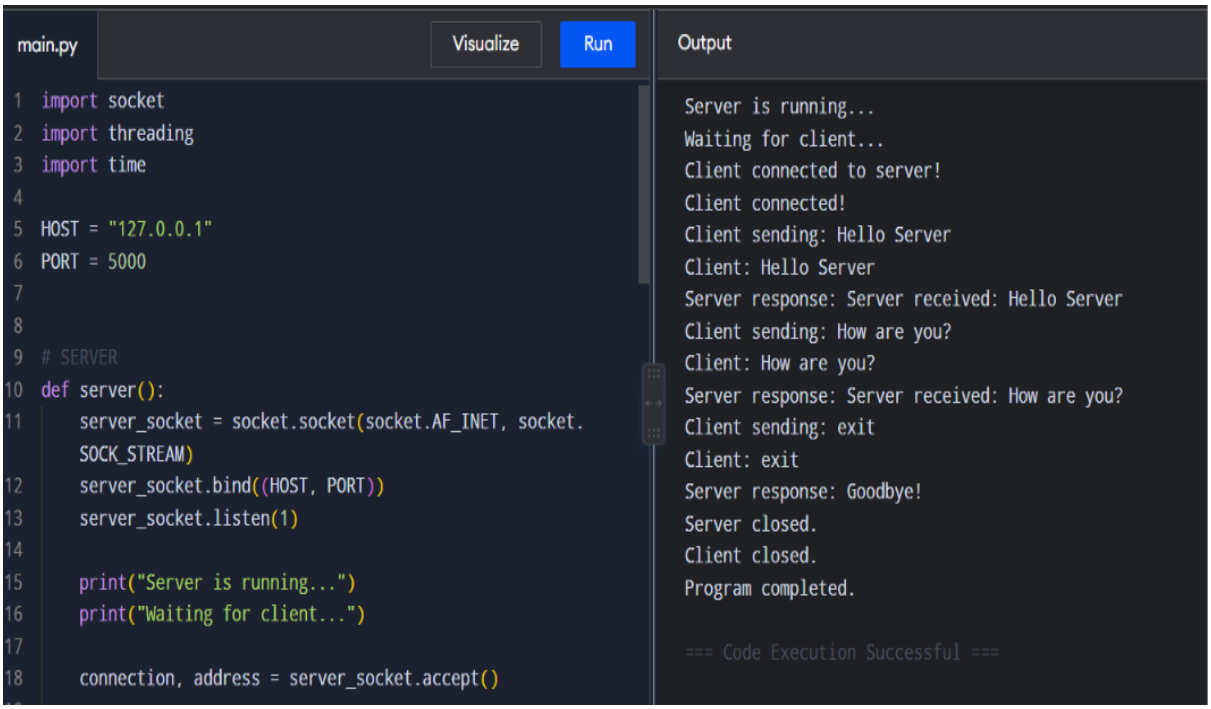
```
import socket
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(("127.0.0.1", 5000))
print("Connected to server.")
while True:
    message = input("Enter message: ")
    client_socket.send(message.encode())
if message == "exit":
        break
 response = client_socket.recv(1024).decode()
    print("Server:", response)
client_socket.close()
```

Working

1. The server creates a TCP socket.
2. It binds the socket to IP address 127.0.0.1 and port 5000.
3. The server listens for a client connection.
4. The client connects to the server.
5. The client enters and sends a message.
6. The server receives and processes the message.
7. The server sends a response to the client.
8. This continues for multiple messages.
9. When the client sends "exit", both sides terminate the connection.

Sample output:

Server:



Question 2 — Reliable Data Transfer over UDP

Aim

To implement reliable data transfer over UDP using **sequence numbers, acknowledgements, timeout and retransmission**.

Sender Program

import socket

sender = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

sender.settimeout(2)

receiver_address = ("127.0.0.1", 6000)

message = input("Enter message: ")

Divide message into packets

packets = [message[i:i+5] for i in range(0, len(message), 5)]

```
for seq, data in enumerate(packets):
    packet = f"{seq}:{data}"
    while True:
        try:
            print("Sending packet:", seq)
            sender.sendto(packet.encode(), receiver_address)
            ack, _ = sender.recvfrom(1024)
            if ack.decode() == f"ACK:{seq}":
                print("ACK received for packet:", seq)
                break
        except socket.timeout:
            print("Timeout! Retransmitting packet:", seq)
sender.sendto(b"END", receiver_address)
print("All packets sent successfully.")
sender.close()
```

Receiver Program

```
import socket
receiver = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
receiver.bind(("127.0.0.1", 6000))
expected_seq = 0
data_packets = {}
print("Receiver is waiting...")
while True:
    packet, address = receiver.recvfrom(1024)
    if packet.decode() == "END":
        break
    packet = packet.decode()
    seq, data = packet.split(":", 1)
```

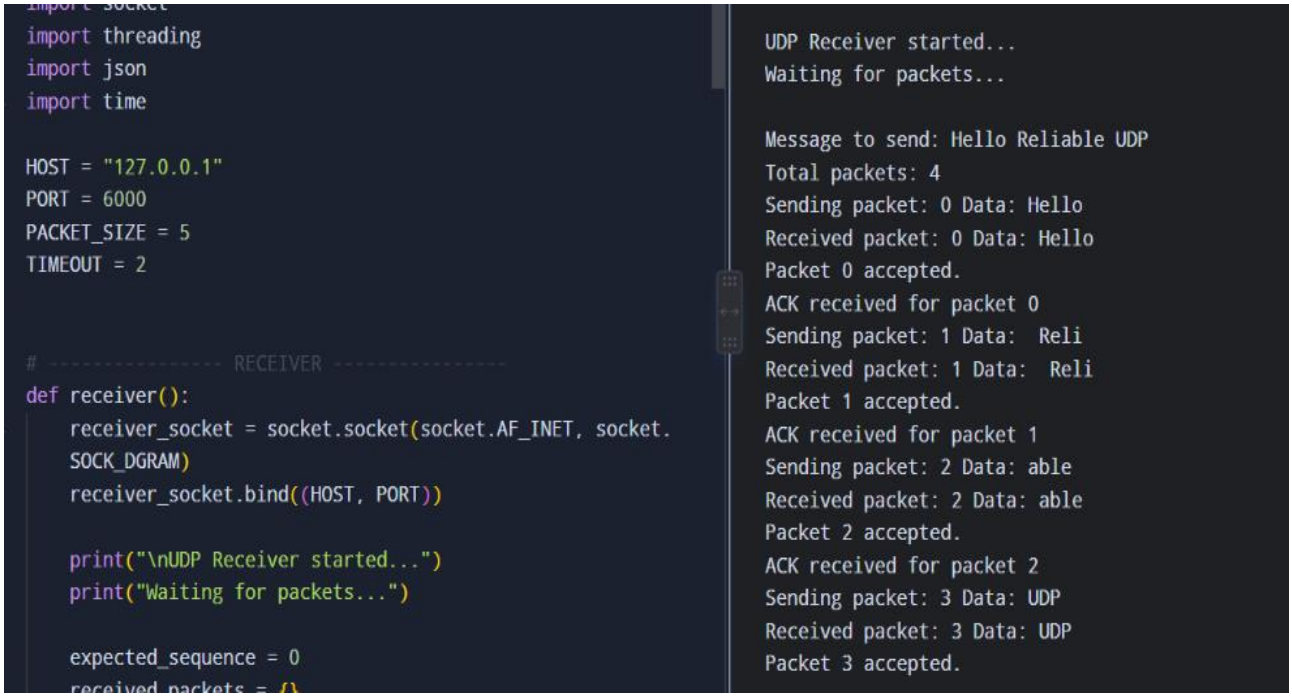
```
    seq = int(seq)
    print("Received packet:", seq)
    if seq == expected_seq:
        data_packets[seq] = data
        receiver.sendto(
            f"ACK:{seq}".encode(),
            address
        )
        expected_seq += 1
    elif seq < expected_seq:
        print("Duplicate packet:", seq)
        receiver.sendto(
            f"ACK:{seq}".encode(),
            address
        )
    else:
        print("Missing packet detected:", expected_seq)
print("Reassembled message:")
message = ""
for seq in sorted(data_packets):
    message += data_packets[seq]
print(message)
receiver.close()
```

Working

1. The sender divides the message into small packets.
2. Each packet is given a sequence number.
3. Packets are transmitted using UDP.
4. The receiver checks the sequence number.
5. If the packet is correct, the receiver sends an ACK.

6. The sender waits for the ACK.
7. If the ACK is not received within the timeout, the sender retransmits the packet.
8. Duplicate packets are detected using sequence numbers.
9. Missing packets can also be detected.
10. Finally, the receiver arranges the packets according to their sequence numbers and reconstructs the original message.

Sample Output



```
import threading
import json
import time

HOST = "127.0.0.1"
PORT = 6000
PACKET_SIZE = 5
TIMEOUT = 2

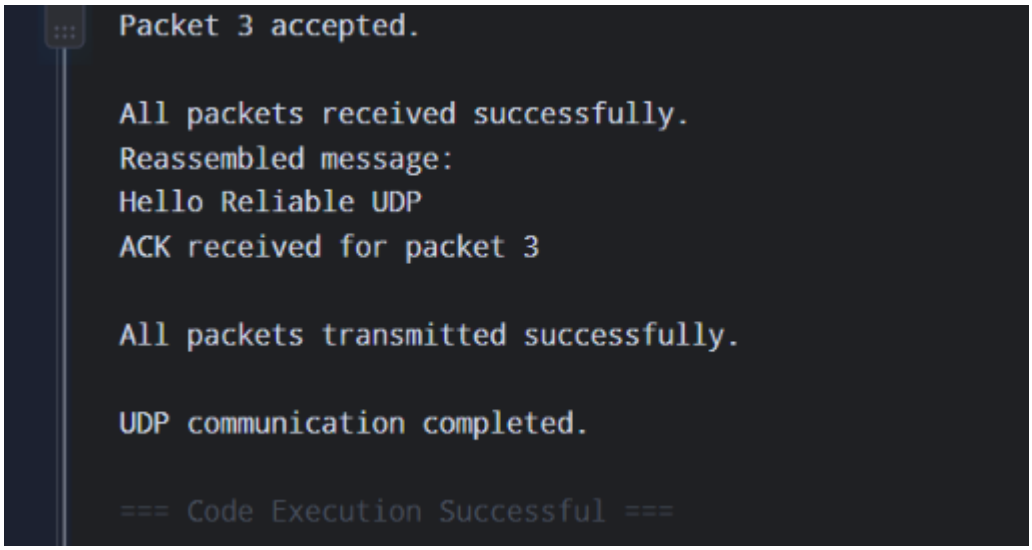

# --------------- RECEIVER ---------------
def receiver():
    receiver_socket = socket.socket(socket.AF_INET, socket.
    SOCK_DGRAM)
    receiver_socket.bind((HOST, PORT))

    print("\nUDP Receiver started...")
    print("Waiting for packets...")

    expected_sequence = 0
```

```
UDP Receiver started...
Waiting for packets...

Message to send: Hello Reliable UDP
Total packets: 4
Sending packet: 0 Data: Hello
Received packet: 0 Data: Hello
Packet 0 accepted.
ACK received for packet 0
Sending packet: 1 Data:  Reli
Received packet: 1 Data:  Reli
Packet 1 accepted.
ACK received for packet 1
Sending packet: 2 Data: able
Received packet: 2 Data: able
Packet 2 accepted.
ACK received for packet 2
Sending packet: 3 Data: UDP
Received packet: 3 Data: UDP
Packet 3 accepted.
```



```
Packet 3 accepted.

All packets received successfully.
Reassembled message:
Hello Reliable UDP
ACK received for packet 3

All packets transmitted successfully.

UDP communication completed.

=== Code Execution Successful ===
```